```
# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like
competitions, datasets, and models to your session
# Learn more about kagglehub:
https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')

# ── 0. INSTALLS
──────────────────────────────────────────────────────────────

!pip install -q transformers

#
================================================================================
=======
#  DINOv2-SMALL: STRUCTURED PRUNING + INT8 QUANTIZATION + SVM
#  88MB → ~15MB  (block dropping + INT8 quant)
#
================================================================================
=======



# ── 1. IMPORTS
──────────────────────────────────────────────────────────────

import os, random, warnings, pickle
import numpy as np
import pandas as pd
from PIL import Image
from tqdm import tqdm
```

```
from copy import deepcopy

import torch
import torch.nn as nn
import torchvision.transforms as T

from transformers import AutoImageProcessor, AutoModel
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

warnings.filterwarnings("ignore")

# ── 2. REPRODUCIBILITY ──────────────────────────────────────────────
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Device: {device}")

# ── 3. DATA LOADING ─────────────────────────────────────────────────
ROOT       = "/kaggle/input/datasets/sumitpr96/cataract-dataset-
combined"
CAT_DIR    = os.path.join(ROOT, "catarct")
NORMAL_DIR = os.path.join(ROOT, "normal")

paths, labels = [], []
for f in os.listdir(CAT_DIR):
    paths.append(os.path.join(CAT_DIR, f)); labels.append(1)
for f in os.listdir(NORMAL_DIR):
    paths.append(os.path.join(NORMAL_DIR, f)); labels.append(0)

df = pd.DataFrame({"path": paths, "label": labels})
train_df, temp_df = train_test_split(df,      test_size=0.30,
stratify=df["label"],      random_state=SEED)
val_df,   test_df = train_test_split(temp_df, test_size=0.50,
stratify=temp_df["label"], random_state=SEED)
print(f"Train: {len(train_df)} | Val: {len(val_df)} | Test:
{len(test_df)}")

# ── 4. TRANSFORMS ───────────────────────────────────────────────────
train_transform = T.Compose([
    T.RandomResizedCrop(224, scale=(0.8, 1.0)),
    T.RandomHorizontalFlip(p=0.5),
    T.RandomRotation(10),
    T.ColorJitter(brightness=0.25, contrast=0.25, saturation=0.15,
hue=0.05),
```

```
    T.GaussianBlur(kernel_size=3, sigma=(0.1, 1.5)),
])
val_transform = T.Compose([T.Resize((224, 224))])

# ── 5. LOAD DINOv2-SMALL & MEASURE ORIGINAL SIZE ──────────────────────
print("\nLoading DINOv2-Small...")
processor = AutoImageProcessor.from_pretrained("facebook/dinov2-
small")
backbone  = AutoModel.from_pretrained("facebook/dinov2-small")

torch.save(backbone.state_dict(), "dinov2_small_original.pth")
orig_size    = os.path.getsize("dinov2_small_original.pth") /
(1024**2)
total_blocks = len(backbone.encoder.layer)
orig_params  = sum(p.numel() for p in backbone.parameters()) / 1e6
print(f"Original  | Blocks: {total_blocks} | Params:
{orig_params:.1f}M | Size: {orig_size:.1f} MB")

# ── 6. STRUCTURED PRUNING: DROP LAST N TRANSFORMER BLOCKS ─────────────
# DINOv2-small has 12 blocks. Each block ≈ 1.77M params ≈ 7MB in FP32
#
# N_BLOCKS_TO_DROP | Blocks kept | FP32 size | INT8 size | Risk
#
#──────────────────┼──────────────┼────────────┼────────────┼──────────────
#        3         |    9 / 12    |  ~67 MB    |  ~17 MB    | low
#        4         |    8 / 12    |  ~60 MB    |  ~15 MB    | moderate ← 
default
#        5         |    7 / 12    |  ~53 MB    |  ~13 MB    | higher
#        6         |    6 / 12    |  ~46 MB    |  ~12 MB    | use only if
acc ok
#
# Later blocks capture global/semantic features; earlier =
texture/edges.
# For binary cataract classification, 8 blocks is usually sufficient.

N_BLOCKS_TO_DROP = 4  # ← change this to tune size vs accuracy
tradeoff

pruned_backbone = deepcopy(backbone)
keep_blocks     = total_blocks - N_BLOCKS_TO_DROP

pruned_backbone.encoder.layer = nn.ModuleList(
    list(pruned_backbone.encoder.layer)[:keep_blocks]
)

pruned_params = sum(p.numel() for p in pruned_backbone.parameters()) /
1e6
print(f"\nPruned    | Blocks: {keep_blocks}/{total_blocks} "
```

```
      f"| Params: {pruned_params:.1f}M dropped {orig_params -
pruned_params:.1f}M")

torch.save(pruned_backbone.state_dict(),
"dinov2_small_pruned_fp32.pth")
pruned_fp32_size = os.path.getsize("dinov2_small_pruned_fp32.pth") /
(1024**2)
print(f"Pruned FP32 size: {pruned_fp32_size:.1f} MB")

# ── 7. INT8 DYNAMIC QUANTIZATION ON PRUNED BACKBONE
──────────────────────────

# Must be done on CPU
print("\nApplying INT8 quantization to pruned backbone...")
pruned_backbone_cpu = deepcopy(pruned_backbone).to("cpu").eval()

quantized_backbone = torch.quantization.quantize_dynamic(
    pruned_backbone_cpu,
    {nn.Linear},
    dtype=torch.qint8
)

torch.save(quantized_backbone.state_dict(),
"dinov2_small_pruned_int8.pth")
int8_size = os.path.getsize("dinov2_small_pruned_int8.pth") /
(1024**2)

print(f"Pruned INT8 size : {int8_size:.1f} MB")
print(f"\nSize journey: {orig_size:.1f} MB → {pruned_fp32_size:.1f} MB
→ {int8_size:.1f} MB")
print(f"Total reduction : {(1 - int8_size/orig_size)*100:.1f}%")

# ── 8. EMBEDDING EXTRACTOR
──────────────────────────────────────────────────

# Use GPU-resident FP32 pruned backbone for fast extraction
# (INT8 model is the deployment artifact — saved above)
embed_model = pruned_backbone.to(device).eval()

@torch.no_grad()
def get_embedding(pil_image):
    inputs  = processor(images=pil_image, return_tensors="pt")
    inputs  = {k: v.to(device) for k, v in inputs.items()}
    outputs = embed_model(**inputs)
    # CLS token at index 0 — 384-dim for DINOv2-small
    return outputs.last_hidden_state[:, 0, :].squeeze().cpu().numpy()

# ── 9. EXTRACT EMBEDDINGS
──────────────────────────────────────────────────

N_AUGS = 8

print("\nExtracting TRAIN embeddings (original + augmented)...")
```

```
X_train, y_train = [], []
for _, row in tqdm(train_df.iterrows(), total=len(train_df)):
    img   = Image.open(row["path"]).convert("RGB")
    label = row["label"]
    # original
    X_train.append(get_embedding(img)); y_train.append(label)
    # augmented
    for _ in range(N_AUGS):
        X_train.append(get_embedding(train_transform(img)));
y_train.append(label)

X_train = np.array(X_train); y_train = np.array(y_train)
print(f"Train: {X_train.shape}")

print("Extracting VAL embeddings...")
X_val, y_val = [], []
for _, row in tqdm(val_df.iterrows(), total=len(val_df)):
    img = Image.open(row["path"]).convert("RGB")
    X_val.append(get_embedding(val_transform(img)));
y_val.append(row["label"])
X_val = np.array(X_val); y_val = np.array(y_val)

print("Extracting TEST embeddings...")
X_test, y_test = [], []
for _, row in tqdm(test_df.iterrows(), total=len(test_df)):
    img = Image.open(row["path"]).convert("RGB")
    X_test.append(get_embedding(val_transform(img)));
y_test.append(row["label"])
X_test = np.array(X_test); y_test = np.array(y_test)

# ── 10. TRAIN SVM ──────────────────────────────────────────────
print("\n" + "="*60)
print("Training SVM (RBF kernel)...")
print("="*60)

svm = SVC(kernel="rbf", C=10, gamma="scale", probability=True,
random_state=SEED)
svm.fit(X_train, y_train)
print("Done.")

# ── 11. EVALUATE ──────────────────────────────────────────────
val_preds  = svm.predict(X_val)
test_preds = svm.predict(X_test)
val_acc    = accuracy_score(y_val,  val_preds)
test_acc   = accuracy_score(y_test, test_preds)

print(f"\n--- Validation ---")
print(f"Accuracy: {val_acc:.4f}")
```

```
print(classification_report(y_val, val_preds, target_names=["Normal",
"Cataract"]))

print(f"--- Test ---")
print(f"Accuracy: {test_acc:.4f}")
print(classification_report(y_test, test_preds,
target_names=["Normal", "Cataract"]))
print("Confusion Matrix:")
print(confusion_matrix(y_test, test_preds))

# ── 12. SAVE SVM
──────────────────────────────────────────────────────────
with open("svm_cataract.pkl", "wb") as f:
    pickle.dump(svm, f)
svm_size = os.path.getsize("svm_cataract.pkl") / (1024**2)

# ── 13. FINAL SUMMARY
─────────────────────────────────────────────────────
print("\n" + "="*60)
print("FINAL SUMMARY")
print("="*60)
print(f"Original DINOv2-Small    : {orig_size:.1f} MB  ({total_blocks}
blocks)")
print(f"After block dropping     : {pruned_fp32_size:.1f} MB
({keep_blocks} blocks, -{N_BLOCKS_TO_DROP} dropped)")
print(f"After INT8 quantization  : {int8_size:.1f} MB  ✓ target
achieved")
print(f"SVM model                : {svm_size:.2f} MB")
print(f"──────────────────────────────────────────")
print(f"Val  Accuracy            : {val_acc:.4f}")
print(f"Test Accuracy            : {test_acc:.4f}")
print(f"Embedding dim            : 384 (unchanged after pruning)")
print(f"──────────────────────────────────────────")
print(f"Deployment files:")
print(f"  dinov2_small_pruned_int8.pth  ({int8_size:.1f} MB) ←
backbone")
print(f"  svm_cataract.pkl              ({svm_size:.2f} MB) ←
classifier")
print("="*60)
```

```
Device: cuda
Train: 1512 | Val: 324 | Test: 325

Loading DINOv2-Small...
```

{"model_id":"80a439cf900240d28ac5b45abcc3387e","version_major":2,"version_minor":0}

```
Original  | Blocks: 12 | Params: 22.1M | Size: 84.2 MB
```

```
Pruned    | Blocks: 8/12 | Params: 15.0M dropped 7.1M
Pruned FP32 size: 57.1 MB

Applying INT8 quantization to pruned backbone...
Pruned INT8 size : 16.6 MB

Size journey: 84.2 MB → 57.1 MB → 16.6 MB
Total reduction : 80.2%

Extracting TRAIN embeddings (original + augmented)...

100%|██████████| 1512/1512 [04:52<00:00,  5.18it/s]

Train: (13608, 384)
Extracting VAL embeddings...

100%|██████████| 324/324 [00:05<00:00, 55.93it/s]

Extracting TEST embeddings...

100%|██████████| 325/325 [00:05<00:00, 56.74it/s]


============================================================
Training SVM (RBF kernel)...
============================================================
Done.

--- Validation ---
Accuracy: 0.9568
              precision    recall  f1-score   support

      Normal       0.95      0.97      0.96       181
    Cataract       0.96      0.94      0.95       143

    accuracy                           0.96       324
   macro avg       0.96      0.95      0.96       324
weighted avg       0.96      0.96      0.96       324

--- Test ---
Accuracy: 0.9631
              precision    recall  f1-score   support

      Normal       0.96      0.97      0.97       181
    Cataract       0.96      0.95      0.96       144

    accuracy                           0.96       325
   macro avg       0.96      0.96      0.96       325
weighted avg       0.96      0.96      0.96       325

Confusion Matrix:
[[176   5]
```

```
 [  7 137]]

============================================================
FINAL SUMMARY
============================================================
Original DINOv2-Small    : 84.2 MB  (12 blocks)
After block dropping     : 57.1 MB  (8 blocks, -4 dropped)
After INT8 quantization  : 16.6 MB  ✓ target achieved
SVM model                : 8.08 MB
------------------------------------------------------------
Val  Accuracy            : 0.9568
Test Accuracy            : 0.9631
Embedding dim            : 384 (unchanged after pruning)
------------------------------------------------------------
Deployment files:
  dinov2_small_pruned_int8.pth  (16.6 MB) ← backbone
  svm_cataract.pkl              (8.08 MB) ← classifier
============================================================

#
# ============================================================================
# =======
#  INFERENCE TEST: INT8 BACKBONE + FP32-TRAINED SVM
#  Tests embedding compatibility between pruned FP32 and INT8 models
#
# ============================================================================
# =======

import os, pickle, warnings
import numpy as np
from PIL import Image
from tqdm import tqdm
from copy import deepcopy

import torch
import torch.nn as nn
import torchvision.transforms as T

from transformers import AutoImageProcessor, AutoModel
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

warnings.filterwarnings("ignore")

# ── 1. LOAD INT8 BACKBONE ──────────────────────────────────────────
# INT8 quantization must run on CPU
print("Loading pruned INT8 backbone...")

# Recreate the pruned architecture (8 blocks) — needed to load the
```

```
state dict
processor   = AutoImageProcessor.from_pretrained("facebook/dinov2-
small")
base_model  = AutoModel.from_pretrained("facebook/dinov2-small")

N_BLOCKS_TO_KEEP = 8   # must match what you used during pruning
base_model.encoder.layer = nn.ModuleList(
    list(base_model.encoder.layer)[:N_BLOCKS_TO_KEEP]
)

# Apply INT8 quantization shell (same as how it was saved)
int8_backbone = torch.quantization.quantize_dynamic(
    base_model.to("cpu").eval(),
    {nn.Linear},
    dtype=torch.qint8
)

# Load the saved INT8 weights
int8_backbone.load_state_dict(
    torch.load("dinov2_small_pruned_int8.pth", map_location="cpu")
)
int8_backbone.eval()
print("INT8 backbone loaded on CPU.")

# ── 2. LOAD SVM (trained on pruned FP32 embeddings) ──────────────
with open("svm_cataract.pkl", "rb") as f:
    svm = pickle.load(f)
print("SVM loaded.")

# ── 3. EMBEDDING EXTRACTOR (INT8, CPU only) ──────────────────────
val_transform = T.Resize((224, 224))

@torch.no_grad()
def get_embedding_int8(pil_image):
    inputs  = processor(images=pil_image, return_tensors="pt")
    # INT8 model is CPU-only — no .to(device) needed
    outputs = int8_backbone(**inputs)
    return outputs.last_hidden_state[:, 0, :].squeeze().cpu().numpy()

# ── 4. EXTRACT TEST EMBEDDINGS USING INT8 BACKBONE ───────────────
print("\nExtracting TEST embeddings using INT8 backbone...")
X_test_int8, y_test = [], []
for _, row in tqdm(test_df.iterrows(), total=len(test_df)):
    img = Image.open(row["path"]).convert("RGB")
    img = val_transform(img)
    X_test_int8.append(get_embedding_int8(img))
    y_test.append(row["label"])
```

```
X_test_int8 = np.array(X_test_int8)
y_test      = np.array(y_test)
print(f"INT8 embeddings shape: {X_test_int8.shape}")

# ── 5. PREDICT WITH SVM ──────────────────────────────────────────
print("\nRunning SVM predictions on INT8 embeddings...")
preds = svm.predict(X_test_int8)
acc   = accuracy_score(y_test, preds)

print(f"\n--- INT8 Backbone + FP32-trained SVM ---")
print(f"Test Accuracy: {acc:.4f}")
print(classification_report(y_test, preds, target_names=["Normal",
"Cataract"]))
print("Confusion Matrix:")
print(confusion_matrix(y_test, preds))

# ── 6. COMPARE FP32 vs INT8 EMBEDDINGS ──────────────────────────
# How different are INT8 embeddings from the FP32 ones the SVM was
trained on?
print("\n--- FP32 vs INT8 Embedding Drift ---")
fp32_preds  = svm.predict(X_test)          # X_test from the original
run
int8_preds  = preds

agreement   = np.mean(fp32_preds == int8_preds)
fp32_acc    = accuracy_score(y_test, fp32_preds)
int8_acc    = accuracy_score(y_test, int8_preds)

print(f"FP32 embedding → SVM accuracy : {fp32_acc:.4f}")
print(f"INT8 embedding → SVM accuracy : {int8_acc:.4f}")
print(f"Accuracy delta                : {abs(fp32_acc -
int8_acc)*100:.2f}%")
print(f"Prediction agreement          : {agreement*100:.2f}% of
samples predicted identically")

# ── 7. DEPLOYMENT SUMMARY ────────────────────────────────────────
int8_size = os.path.getsize("dinov2_small_pruned_int8.pth") /
(1024**2)
svm_size  = os.path.getsize("svm_cataract.pkl")             /
(1024**2)

print("\n" + "="*60)
print("DEPLOYMENT SUMMARY")
print("="*60)
print(f"INT8 backbone          : {int8_size:.1f} MB")
print(f"SVM classifier         : {svm_size:.2f} MB")
```

```
print(f"Total deployment size  : {int8_size + svm_size:.1f} MB")
print(f"Test accuracy (INT8)   : {int8_acc:.4f}")
print(f"Accuracy drop vs FP32  : {abs(fp32_acc - int8_acc)*100:.2f}%")
print("="*60)
```

```
Loading pruned INT8 backbone...
```


```
{"model_id":"f84d17b102c5435d962025b0d3890ab4","version_major":2,"vers
ion_minor":0}
```


```
INT8 backbone loaded on CPU.
SVM loaded.

Extracting TEST embeddings using INT8 backbone...

100%|██████████| 325/325 [00:33<00:00,  9.76it/s]

INT8 embeddings shape: (325, 384)

Running SVM predictions on INT8 embeddings...

--- INT8 Backbone + FP32-trained SVM ---
Test Accuracy: 0.9262
              precision    recall  f1-score   support

      Normal       0.95      0.91      0.93       181
    Cataract       0.89      0.94      0.92       144

    accuracy                           0.93       325
   macro avg       0.92      0.93      0.93       325
weighted avg       0.93      0.93      0.93       325

Confusion Matrix:
[[165  16]
 [  8 136]]

--- FP32 vs INT8 Embedding Drift ---
FP32 embedding → SVM accuracy : 0.9631
INT8 embedding → SVM accuracy : 0.9262
Accuracy delta                : 3.69%
Prediction agreement          : 95.08% of samples predicted
identically

============================================================
DEPLOYMENT SUMMARY
============================================================
INT8 backbone          : 16.6 MB
SVM classifier         : 8.08 MB
Total deployment size  : 24.7 MB
Test accuracy (INT8)   : 0.9262
```

```
Accuracy drop vs FP32  : 3.69%
============================================================
```

```python
import os
import random
import warnings
import pickle
import numpy as np
import pandas as pd
from PIL import Image
from tqdm import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import torchvision.transforms as T
from transformers import AutoImageProcessor, AutoModel

from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

warnings.filterwarnings("ignore")

# ========================================
# 1. SETUP & REPRODUCIBILITY
# ========================================
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# ========================================
# 2. DATA LOADING & SPLITTING
# ========================================
ROOT = "/kaggle/input/datasets/sumitpr96/cataract-dataset-combined"
CAT_DIR = os.path.join(ROOT, "catarct")
NORMAL_DIR = os.path.join(ROOT, "normal")

paths, labels = [], []
print("Loading Data Paths...")
for f in os.listdir(CAT_DIR):
    paths.append(os.path.join(CAT_DIR, f))
    labels.append(1)
```

```
for f in os.listdir(NORMAL_DIR):
    paths.append(os.path.join(NORMAL_DIR, f))
    labels.append(0)

df = pd.DataFrame({"path": paths, "label": labels})
train_df, temp_df = train_test_split(df, test_size=0.30,
stratify=df["label"], random_state=SEED)
val_df, test_df   = train_test_split(temp_df, test_size=0.50,
stratify=temp_df["label"], random_state=SEED)
print(f"Train: {len(train_df)} | Val: {len(val_df)} | Test:
{len(test_df)}")

# ========================================
# 3. TRANSFORMS & DATALOADERS
# ========================================
train_transform = T.Compose([
    T.RandomResizedCrop(224, scale=(0.8, 1.0)),
    T.RandomHorizontalFlip(p=0.5),
    T.RandomRotation(10),
    T.ColorJitter(brightness=0.25, contrast=0.25, saturation=0.15,
hue=0.05),
    T.GaussianBlur(kernel_size=3, sigma=(0.1, 1.5)),
])
val_transform = T.Compose([T.Resize((224, 224))])

processor = AutoImageProcessor.from_pretrained("facebook/dinov2-
small")

class CataractDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform
    def __len__(self):
        return len(self.df)
    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img = Image.open(row["path"]).convert("RGB")
        if self.transform:
            img = self.transform(img)
        inputs = processor(images=img, return_tensors="pt")
        return inputs['pixel_values'].squeeze(0),
torch.tensor(row["label"], dtype=torch.long)

train_dataset = CataractDataset(train_df, transform=train_transform)
train_loader  = DataLoader(train_dataset, batch_size=16, shuffle=True)

# ========================================
# 4. RUNTIME INTROSPECTION HELPERS
# ========================================
```

```
def _get_attr(obj, *candidates):
    for name in candidates:
        if hasattr(obj, name):
            return name, getattr(obj, name)
    raise AttributeError(
        f"{type(obj).__name__} has none of the expected attributes:
{candidates}\n"
        f"Actual attributes: {[k for k in obj._modules] + [k for k in
obj.__dict__]}"
    )

def _resolve_layer_norms(layer):
    before_name, _ = _get_attr(layer, "layernorm_before", "norm1",
"layer_norm1")
    after_name,  _ = _get_attr(layer, "layernorm_after",  "norm2",
"layer_norm2")
    return before_name, after_name

def _resolve_mlp_parts(mlp):
    _, fc1 = _get_attr(mlp, "fc1")
    _, act = _get_attr(mlp, "activation", "act_fn",
"intermediate_act_fn")
    _, fc2 = _get_attr(mlp, "fc2")
    return fc1, act, fc2


# ========================================
# 5. QUANTIZABLE MLP WRAPPER
# ========================================
class QuantizableMLP(nn.Module):
    def __init__(self, mlp_module):
        super().__init__()
        self.fc1, self.act, self.fc2 = _resolve_mlp_parts(mlp_module)
        self.quant   = torch.ao.quantization.QuantStub()
        self.dequant = torch.ao.quantization.DeQuantStub()

    def forward(self, x):
        x = self.quant(x)
        x = self.fc1(x)
        x = self.act(x)
        x = self.fc2(x)
        x = self.dequant(x)
        return x


def patch_encoder_mlps(encoder):
    for layer in encoder.layer:
        norm_before_name, norm_after_name =
_resolve_layer_norms(layer)
        layer.mlp = QuantizableMLP(layer.mlp)
```

```
        def make_forward(blk, nb, na):
            def forward(hidden_states):
                norm_before = getattr(blk, nb)
                norm_after  = getattr(blk, na)
                hidden_states_norm = norm_before(hidden_states)
                attn_out = blk.attention(hidden_states_norm)
                attn_out = blk.layer_scale1(attn_out)
                hidden_states = hidden_states + attn_out
                hidden_states_norm = norm_after(hidden_states)
                mlp_out = blk.mlp(hidden_states_norm)
                mlp_out = blk.layer_scale2(mlp_out)
                hidden_states = hidden_states + mlp_out
                return hidden_states
            return forward

        layer.forward = make_forward(layer, norm_before_name,
norm_after_name)
    return encoder


# ============================================
# 6. MAIN MODEL
# ============================================
class QATDinoClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        base_model = AutoModel.from_pretrained(
            "facebook/dinov2-small",
            attn_implementation="eager"
        )
        self.embeddings = base_model.embeddings
        self.encoder    = base_model.encoder
        self.encoder.layer = nn.ModuleList(list(self.encoder.layer)
[:8])
        patch_encoder_mlps(self.encoder)
        self.layernorm  = base_model.layernorm
        self.classifier = nn.Linear(384, 2)

    def _encode(self, pixel_values):
        x = self.embeddings(pixel_values)
        encoder_out = self.encoder(x)
        x = self.layernorm(encoder_out[0])
        return x[:, 0, :]

    def forward(self, pixel_values):
        return self.classifier(self._encode(pixel_values))

    def extract_features(self, pixel_values):
        return self._encode(pixel_values)
```

```
print("\nInitializing QAT Model...")
model = QATDinoClassifier().to(device)

model.qconfig =
torch.ao.quantization.get_default_qat_qconfig('fbgemm')
model.embeddings.qconfig = None
model.layernorm.qconfig   = None
model.classifier.qconfig  = None

for layer in model.encoder.layer:
    layer.attention.qconfig    = None
    layer.layer_scale1.qconfig = None
    layer.layer_scale2.qconfig = None
    nb, na = _resolve_layer_norms(layer)
    getattr(layer, nb).qconfig = None
    getattr(layer, na).qconfig = None

model.train()
torch.ao.quantization.prepare_qat(model, inplace=True)

# ==========================================
# 7. QUANTIZATION-AWARE TRAINING
# ==========================================
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-5)
EPOCHS = 2

print("\nStarting Quantization-Aware Training (QAT)...")
for epoch in range(EPOCHS):
    model.train()
    loop = tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS}")
    for images, batch_labels in loop:
        images, batch_labels = images.to(device),
batch_labels.to(device)
        optimizer.zero_grad()
        loss = criterion(model(images), batch_labels)
        loss.backward()
        optimizer.step()
        loop.set_postfix(loss=loss.item())

# ==========================================
# 8. EXTRACT EMBEDDINGS ON GPU
#
# INT8 convert() requires CPU (no QuantizedCUDA backend exists).
# We stay on GPU in QAT eval mode for embedding extraction — fake-
quant
# eval is functionally equivalent and gives full GPU throughput.
# INT8 conversion happens AFTER embeddings are done, only for saving.
```

```
# ============================================
print("\nSetting eval mode for embedding extraction (model stays on
GPU)...")
model.eval()

# Also batch the augmented train embeddings for better GPU utilisation
BATCH_SIZE = 64  # process this many augmented images at once

@torch.no_grad()
def get_embeddings_batched(pil_images):
    """Process a list of PIL images in one GPU batch."""
    inputs = processor(images=pil_images, return_tensors="pt")
    pixel_values = inputs['pixel_values'].to(device)          # <--
GPU
    return model.extract_features(pixel_values).cpu().numpy() # back
to CPU/numpy

print("\nExtracting TRAIN embeddings (GPU, QAT eval mode +
Augmentations)...")
N_AUGS = 8
X_train_qat, y_train = [], []

for _, row in tqdm(train_df.iterrows(), total=len(train_df)):
    img   = Image.open(row["path"]).convert("RGB")
    label = row["label"]

    # Collect base image + N_AUGS augmented copies, then forward in
one batch
    batch_imgs = [img] + [train_transform(img) for _ in range(N_AUGS)]
    embeddings = get_embeddings_batched(batch_imgs)  # (N_AUGS+1, 384)

    X_train_qat.extend(embeddings)
    y_train.extend([label] * len(batch_imgs))

X_train_qat = np.array(X_train_qat)
y_train     = np.array(y_train)

print("\nExtracting TEST embeddings (GPU)...")
X_test_qat, y_test = [], []

for _, row in tqdm(test_df.iterrows(), total=len(test_df)):
    img = Image.open(row["path"]).convert("RGB")
    img = val_transform(img)
    emb = get_embeddings_batched([img])             # (1, 384)
    X_test_qat.append(emb[0])
    y_test.append(row["label"])

X_test_qat = np.array(X_test_qat)
y_test     = np.array(y_test)
```

```
# ==========================================
# 9. CONVERT TO INT8 (CPU, for saving only)
# ==========================================
print("\nConverting MLP layers to INT8 for deployment saving
(CPU)...")
model.to("cpu")
torch.ao.quantization.convert(model, inplace=True)

# ==========================================
# 10. PCA COMPRESSION
# ==========================================
print(f"\nOriginal embedding dimensions: {X_train_qat.shape[1]}")
print("Fitting PCA to reduce 384D -> 64D...")
pca = PCA(n_components=64, random_state=SEED)
X_train_pca = pca.fit_transform(X_train_qat)
X_test_pca  = pca.transform(X_test_qat)
print(f"Compressed train shape: {X_train_pca.shape}")

# ==========================================
# 11. SVM TRAINING & EVALUATION
# ==========================================
print("\nTraining final RBF SVM on 64D embeddings...")
final_svm = SVC(kernel="rbf", C=10, gamma="scale", probability=True,
random_state=SEED)
final_svm.fit(X_train_pca, y_train)

print("\nEvaluating Final Deployed Pipeline (QAT INT8 MLPs -> PCA ->
SVM)...")
preds     = final_svm.predict(X_test_pca)
final_acc = accuracy_score(y_test, preds)

print(f"\nTest Accuracy: {final_acc:.4f}")
print(classification_report(y_test, preds, target_names=["Normal",
"Cataract"]))
print("Confusion Matrix:")
print(confusion_matrix(y_test, preds))

# ==========================================
# 12. SAVE FINAL ARTIFACTS
# ==========================================
torch.save(model.state_dict(), "dinov2_8layer_qat_int8.pth")
with open("pca_transformer.pkl", "wb") as f:
    pickle.dump(pca, f)
with open("svm_qat_pca.pkl", "wb") as f:
    pickle.dump(final_svm, f)

int8_size = os.path.getsize("dinov2_8layer_qat_int8.pth") / (1024**2)
pca_size  = os.path.getsize("pca_transformer.pkl")         / (1024**2)
svm_size  = os.path.getsize("svm_qat_pca.pkl")             / (1024**2)
```

```python
print("\n" + "="*50)
print("FINAL DEPLOYMENT SIZE")
print("="*50)
print(f"INT8 Backbone : {int8_size:.2f} MB")
print(f"PCA Component : {pca_size:.2f} MB")
print(f"SVM Classifier: {svm_size:.2f} MB")
print("-"*50)
print(f"Total Size    : {int8_size + pca_size + svm_size:.2f} MB")
print("="*50)
```

```
Using device: cuda
Loading Data Paths...
Train: 1512 | Val: 324 | Test: 325

Initializing QAT Model...
```

{"model_id":"85c6be637110409c9456cc9299d73a63","version_major":2,"version_minor":0}

```
Starting Quantization-Aware Training (QAT)...

Epoch 1/2: 100%|██████████| 95/95 [00:44<00:00,  2.14it/s, loss=0.0447]
Epoch 2/2: 100%|██████████| 95/95 [00:42<00:00,  2.25it/s, loss=0.00459]

Setting eval mode for embedding extraction (model stays on GPU)...

Extracting TRAIN embeddings (GPU, QAT eval mode + Augmentations)...

100%|██████████| 1512/1512 [03:47<00:00,  6.65it/s]

Extracting TEST embeddings (GPU)...

100%|██████████| 325/325 [00:07<00:00, 43.54it/s]

Converting MLP layers to INT8 for deployment saving (CPU)...

Original embedding dimensions: 384
Fitting PCA to reduce 384D -> 64D...
Compressed train shape: (13608, 64)

Training final RBF SVM on 64D embeddings...

Evaluating Final Deployed Pipeline (QAT INT8 MLPs -> PCA -> SVM)...

Test Accuracy: 0.9631
              precision    recall  f1-score   support
```

```
      Normal       0.95      0.98      0.97       181
    Cataract       0.98      0.94      0.96       144

    accuracy                           0.96       325
   macro avg       0.97      0.96      0.96       325
weighted avg       0.96      0.96      0.96       325

Confusion Matrix:
[[178   3]
 [  9 135]]

==================================================
FINAL DEPLOYMENT SIZE
==================================================
INT8 Backbone : 30.37 MB
PCA Component : 0.10 MB
SVM Classifier: 0.56 MB
--------------------------------------------------
Total Size    : 31.02 MB
==================================================
```

```python
import os
import zipfile
from IPython.display import FileLink, display

# Only the 3 model files (no processor config, to avoid external
downloads)
model_files = [
    "dinov2_8layer_qat_int8.pth",
    "pca_transformer.pkl",
    "svm_qat_pca.pkl"
]

print("Checking files...")
missing = []
for f in model_files:
    if os.path.exists(f):
        size = os.path.getsize(f) / (1024 * 1024)
        print(f"□ {f}: {size:.2f} MB")
    else:
        print(f"□ {f} not found!")
        missing.append(f)

if missing:
    print("\n⚠ Please run the QAT training cell first to generate the
missing files.")
else:
    # Create ZIP
    zip_name = "cataract_qat_deployment.zip"
```

```
    print(f"\n Creating {zip_name}...")
    with zipfile.ZipFile(zip_name, 'w', zipfile.ZIP_DEFLATED) as zipf:
        for f in model_files:
            zipf.write(f)

    total_size = os.path.getsize(zip_name) / (1024 * 1024)
    print(f" Done! ZIP size: {total_size:.2f} MB")

    # Download link
    print("\n Click below to download:")
    display(FileLink(zip_name))

    print(f"\n Manual path: /kaggle/working/{zip_name}")
```

```
Checking files...
 dinov2_8layer_qat_int8.pth: 30.37 MB
 pca_transformer.pkl: 0.10 MB
 svm_qat_pca.pkl: 0.56 MB

 Creating cataract_qat_deployment.zip...
 Done! ZIP size: 28.05 MB

 Click below to download:

/kaggle/working/cataract_qat_deployment.zip


 Manual path: /kaggle/working/cataract_qat_deployment.zip
```

```
import os
import json

# Create the folder
os.makedirs("dinov2_processor_config", exist_ok=True)

# The exact standard config for DINOv2-small (ImageNet stats)
config = {
    "do_resize": True,
    "do_rescale": True,
    "image_mean": [0.485, 0.456, 0.406],
    "image_std": [0.229, 0.224, 0.225],
    "resample": 3,  # BICUBIC
    "size": {"height": 224, "width": 224},
    "rescale_factor": 0.00392156862745098  # 1/255
}

# Write it to the folder
with open("dinov2_processor_config/preprocessor_config.json", "w") as
f:
    json.dump(config, f, indent=2)
```

```
print("□ processor config created locally (0.7 KB)")
```

```
□ processor config created locally (0.7 KB)
```

```
import os
import zipfile
from IPython.display import FileLink, display

model_files = [
    "dinov2_8layer_qat_int8.pth",
    "pca_transformer.pkl",
    "svm_qat_pca.pkl"
]

zip_name = "cataract_qat_deployment_full.zip"

with zipfile.ZipFile(zip_name, 'w', zipfile.ZIP_DEFLATED) as zipf:
    # Add the 3 model files
    for f in model_files:
        if os.path.exists(f):
            zipf.write(f)

    # Add the manually created config folder
    if os.path.exists("dinov2_processor_config"):
        for root, dirs, files in os.walk("dinov2_processor_config"):
            for file in files:
                zipf.write(os.path.join(root, file),
                           os.path.join("dinov2_processor_config",
file))

total_size = os.path.getsize(zip_name) / (1024 * 1024)
print(f"□ Full deployment ZIP created: {zip_name} ({total_size:.2f}
MB)")
display(FileLink(zip_name))
```

```
□ Full deployment ZIP created: cataract_qat_deployment_full.zip (28.05
MB)
```

```
/kaggle/working/cataract_qat_deployment_full.zip
```